Session 18 Teamwork — A New Table, a New Index, and a New Way to Lock a Row

Context

Work in your team (six teams, each team of four students). Live code built a real index (`idx_issues_status`) on the existing `issues` table, and proved a real atomic transaction pattern (`BEGIN` / `UPDATE ... RETURNING` / `INSERT` / `COMMIT`, plus a real `ROLLBACK`) for generating a real project's next issue key (`WEB-3`). Nobody has shown you, live, a table with more than one column in its own index, or any way to do the identical atomic key-generation job using an explicit row lock instead.

Goal

A new real table, `activity_logs` — the relational teaching subset of this project's own frozen domain model needed for this session (`projects/b4f-board/specification/DATABASE-PROGRESSION.md`'s own Activity History entry: its three foreign keys, an `action` description, and a timestamp; the frozen model's own `metadata` field and constrained `action` vocabulary are deliberately not built here — deferred to the real ORM migration) — with two new real compound (multicolumn) indexes you research and build yourselves, plus a real, proven alternative to the live-taught transaction pattern using `SELECT ... FOR UPDATE` — researched and implemented by your team, then proven with your own real, live, two-terminal-window demonstration.

What to build

You already know `CREATE INDEX` (one column) and `BEGIN` / `COMMIT` / `ROLLBACK`. Research, then build:

- **`activity_logs`**: references a workspace (`workspace_id`), optionally an issue (`issue_id`), and optionally an actor (`actor_id`, a real reference to `users`), plus a real `action` description and timestamp — matching the real domain model this program already uses.
- **`ON DELETE`.** You already learned this real mechanism last session — reuse it, do not re-research it: `workspace_id` and `issue_id` are ownership-hierarchy references (the same real category as `issues.project_id`); `actor_id` is a user-reference (the same real category as `issues.assignee_id` / `reporter_id`).
- **Two compound indexes** — research "multicolumn indexes" in the real official PostgreSQL documentation. Build one matching "most recent activity for a given issue" and one matching "most recent activity for a given workspace." Think carefully about which column goes first.
- **`SELECT ... FOR UPDATE`** — research this in the real official PostgreSQL documentation on explicit row locking. Use it to generate a real new issue key for a DIFFERENT real project than the one shown live (not WEB) — the identical atomic read-increment-use job `UPDATE ... RETURNING` already did live, solved a different real way.
- **A real, live proof that the lock actually blocks**, using two of your own real terminal windows at once (see "Test it" below).

- **Written synthesis required:** which category each of `activity_logs`'s own three foreign keys belongs to and why (one real sentence each); what `SELECT ... FOR UPDATE` actually locks and for how long; one real, concrete reason you might prefer it over `UPDATE ... RETURNING` even though both reach the identical result here.

Rules

Allowed: everything already shown live, plus everything you discover by reading the real official PostgreSQL documentation on multicolumn indexes and on explicit row locking (`SELECT ... FOR UPDATE`) — real, documented, standard SQL mechanisms, nothing invented from scratch. `psql`'s own `\d+ tablename` to inspect a table's real constraints and indexes directly.

Not yet: no named transaction isolation levels as formal theory — the plain default behavior is enough for this task; no window functions; no `metadata` column or enforced `action` vocabulary on `activity_logs` (deferred to the real ORM migration).

Forbidden: do not alter `users`, `workspaces`, `workspace_members`, `projects`, `issues`, `comments`, or any of their existing constraints — including their `ON DELETE` clauses, even if your research suggests one should change. This teamwork is scoped entirely to `activity_logs`: its own table, its own foreign keys, its own indexes, and the `SELECT ... FOR UPDATE` proof. Session 17's delivered schema is not yours to "correct" during teamwork.

Test it (design your own full proof — this is a starting point, not the complete list)

Re-seed a fresh database with `schema.sql`, then the live-code seed inserts, then your own team's `activity_logs` seed rows — include a real bulk-seeded set (at least a few thousand rows) so your compound indexes have real data to prove themselves against with `EXPLAIN ANALYZE`.

macOS terminal (only if not already connected from live code today)

BASH

```
psql -d <database>
```

inside psql — <database>=>

Prove your compound indexes are actually used:

SQL

```
EXPLAIN ANALYZE SELECT id, action, created_at FROM activity_logs WHERE workspace_id = <real
id> ORDER BY created_at DESC LIMIT 20;
```

Prove the `ON DELETE` behavior you chose for each foreign key, the same way you proved it last session.

For the real lock-blocking proof, you need a SECOND real terminal window — open a new one, connect again (`psql -d <database>`), and run the identical `SELECT ... FOR UPDATE` statement there while the FIRST window's own transaction is still open (not yet committed). Watch what happens.

Done when

- `activity_logs` exists, matching this session's own teaching subset of the frozen domain model, with all three foreign keys correctly classified and proven with a real `DELETE`.
- Both compound indexes exist and are proven, with real `EXPLAIN ANALYZE` output, to actually be used.
- A real new issue exists for a project other than WEB, generated via `SELECT ... FOR UPDATE`.
- Your team has actually run the real two-window concurrency proof themselves and can explain, out loud, what they observed.
- Your written synthesis covers all three required points above.

Working time

About 60–70 minutes — researching and correctly implementing `SELECT ... FOR UPDATE`, and actually running the real two-window proof, both deserve genuine time; this is new, real material nobody has shown you yet.